

Introduction to Secure Communication

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



July 27 , 2026

Table of Content

- 1 Introduction to Secure Communication
- 2 Achieving Confidentiality
- 3 Example Constructions
- 4 Establishing Short Key
- 5 Achieving Integrity
- 6 Achieving Authenticity

Recommended Reading From textbook: Chapter 3.1-3.6

Introduction to Secure Communication

Secure Communication: The Basic Scenario

Alice $\Rightarrow$ Bob

- Alice wants to send a message to Bob.
- The communication channel is public.
- Anyone may observe the transmitted data.
- Goal: Bob should receive the intended message correctly.

Sending a Message over a Public Channel

Alice

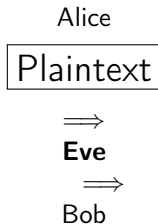
Plaintext: "Meet at 5 PM"

⇒ Public Network ⇒

Bob

- The message is transmitted in plaintext.
- Every intermediate node can read the message.
- No confidentiality is provided.

An Adversary Appears



- Eve can eavesdrop on the communication.
- She learns the complete message.
- Privacy is completely lost.

Encrypt Before Transmission

$$\text{Ciphertext} = \text{Enc}_K(\text{Plaintext})$$

Alice

Ciphertext



Eve



Bob

$$\text{Plaintext} = \text{Dec}_K(\text{Ciphertext})$$

What Should Security Achieve?

Alice $\implies$ Ciphertext $\implies$ Bob

Even if Eve intercepts the ciphertext,

Ciphertext reveals no useful information about the plaintext

A secure communication system should provide:

- Confidentiality
- Integrity
- Authentication
- Availability (system level)

Fundamental Security Goals

1 Confidentiality

- Only authorized parties should be able to access the information.

2 Integrity

- Data should not be altered without detection during storage or transmission.

3 Authentication

- Verify the identity of communicating parties and the origin of the data.

Availability

- Information and services should remain accessible to authorized users when needed.

Achieving Confidentiality

Achieving Confidentiality

Goal: Prevent unauthorized parties from learning the message.

Plaintext $\xrightarrow{\text{Enc}_K}$ Ciphertext $\xrightarrow{\text{Public Channel}}$ Ciphertext $\xrightarrow{\text{Dec}_K}$ Plaintext

- Encryption transforms readable data into an unintelligible form.
- Only authorized users can recover the original message.

Encryption and Decryption

Let

m = message, c = ciphertext, k = secret key.

Encryption:

$$c = \text{Enc}_k(m)$$

Decryption:

$$m = \text{Dec}_k(c)$$

Correctness requires

$$\text{Dec}_k(\text{Enc}_k(m)) = m.$$

Examples Encryption Schemes (Symmetric-Key)

Scheme	Type	Typical Key Size
DES	Block Cipher	56 bits
3DES	Block Cipher	112/168 bits
AES	Block Cipher	128/192/256 bits
ChaCha20	Stream Cipher	256 bits
Salsa20	Stream Cipher	256 bits

- AES is the current standard for symmetric encryption.
- DES is no longer considered secure due to its short key length.
- ChaCha20 is widely used in software and mobile applications.

Symmetric-Key Encryption

Alice $\longleftrightarrow$ Bob
Shared Secret Key k

Encryption:

$$c = \text{Enc}_k(m)$$

Decryption:

$$m = \text{Dec}_k(c)$$

- Alice and Bob use the same secret key k .
- The security of the scheme relies on keeping k secret.
- Anyone knowing k can both encrypt and decrypt messages.

The Key Distribution Problem

Symmetric encryption assumes that Alice and Bob already share a secret key.

How can Alice and Bob establish k securely?

Possible solutions:

- Physical exchange of the key.
- Trusted third party.
- Public-key cryptography.

Public-Key Encryption

Each user possesses a pair of keys:

$$(pk, sk)$$

- Public key (pk): Shared with everyone.
- Secret key (sk): Kept private by the owner.

Encryption:

$$c = \text{Enc}_{pk}(m)$$

Decryption:

$$m = \text{Dec}_{sk}(c)$$

Symmetric vs Public-Key Encryption

	Symmetric	Public-Key
Encryption Key	Secret key k	Public key pk
Decryption Key	Secret key k	Secret key sk
Speed	Fast	Slower
Key Distribution	Difficult	Easier
Typical Use	Data Encryption	Key Exchange

- Modern secure communication protocols combine both approaches.

Roadmap

So far, we have seen *what* encryption achieves. Next, we study *how* encryption schemes are constructed mathematically.

The next part of the course will cover:

- Mathematical model of an encryption scheme.
- Security definitions for encryption.
- Construction of symmetric-key encryption schemes.
- Key distribution: How two parties establish a shared secret key.

Encryption Scheme

A symmetric-key encryption scheme consists of three algorithms:

(Gen, Enc, Dec)

Key generation:

The key generation algorithm outputs a secret key.

$$k \leftarrow \text{Gen}(1^\lambda)$$

- λ is the security parameter.
- k is known only to legitimate users.

Encryption:

$$c \leftarrow \text{Enc}_k(m), \quad m \in \mathcal{M} : \text{message space}$$

Decryption:

$$m \leftarrow \text{Dec}_k(c), \quad c \in \mathcal{C} : \text{ciphertext space}$$

An encryption scheme must satisfy

$$\text{Dec}_k(\text{Enc}_k(m)) = m$$

for every

$$k \leftarrow \text{Gen}(1^\lambda), \quad m \in \mathcal{M}.$$

Example Construction of Symmetric key Encryption Scheme.

Caesar Cipher $\implies$ Monoalphabetic Cipher $\implies$ Vigenère Cipher $\implies$
One-Time Pad

Each scheme attempts to overcome a weakness of its predecessor:

- Caesar: Very small key space.
- Monoalphabetic: Vulnerable to frequency analysis.
- Vigenère: Hides frequencies using multiple alphabets.
- One-Time Pad: Achieves perfect secrecy.

Caesar Cipher

Each letter is shifted by a fixed amount k .

$$c = (m + k) \bmod 26$$

$$m = (c - k) \bmod 26$$

Example ($k = 3$):

HELLO $\rightarrow$ KHOOR

Problem:

- Only 25 possible keys.
- Easily broken by brute force.

Monoalphabetic Substitution Cipher

Each plaintext letter is replaced by a fixed permutation.

$$c = \pi(m), \quad m = \pi^{-1}(c)$$

where π is a permutation of the alphabet.

Example:

$$A \mapsto Q, \quad B \mapsto X, \quad C \mapsto M, \dots$$

Improvement:

Much larger key space.

Problem:

- Letter frequencies are preserved.
- Broken using frequency analysis.

Key Space of the Monoalphabetic Substitution Cipher

Each plaintext letter is mapped to a unique ciphertext letter.

The secret key is a permutation of the alphabet.

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>...</i>	<i>Y</i>	<i>Z</i>
↓	↓	↓	↓	↓		↓	↓
<i>Q</i>	<i>L</i>	<i>M</i>	<i>X</i>	<i>P</i>	<i>...</i>	<i>C</i>	<i>H</i>

Number of possible keys:

$$26! \approx 4.03 \times 10^{26} \approx 2^{88}$$

- Far larger than the Caesar cipher (25 keys).
- Exhaustive search is computationally infeasible.

Why Does Frequency Analysis Still Work?

Although the substitution changes the letters, it does *not* change their frequencies.

Plaintext Letter	E	T	A	O	N	...
Frequency	High	High	High	High	High	
Ciphertext Letter	X	Q	M	J	K	...
Frequency	High	High	High	High	High	

An attacker compares ciphertext letter frequencies with known language statistics to recover the substitution.

Large key space alone does not guarantee security.

Language Statistics Differ Across Languages

Letter frequencies are characteristic of a language and are remarkably stable over large texts.

Rank	1	2	3	4	5
English	E	T	A	O	I
French	E	A	S	I	T

Common words also reveal the language.

English	the, and, of, to, in
French	le, de, et, la, les

Frequency analysis exploits language statistics, not just the cipher.

How to Fix it?

Vigenère Cipher

Use multiple Caesar ciphers controlled by a keyword.

For keyword $k_1 k_2 \dots k_t$,

$$c_i = (m_i + k_i) \bmod 26.$$

Example:

ATTACKATDAWN

Keyword:

LEMONLEMONLE

Improvement:

Letter frequencies are spread across multiple alphabets.

Problem:

- Repeated key introduces patterns.
- Broken using Kasiski and Friedman attacks.

How Does the Vigenère Cipher Improve Security?

The Vigenère cipher uses multiple Caesar ciphers instead of a single substitution.

$$\text{Ciphertext}_i = (m_i + k_i) \bmod 26$$

where the key is repeated periodically.

Position	1	2	3	4	5	6	7	8
Key	L	E	M	O	N	L	E	M

Consequences:

- The same plaintext letter may encrypt to different ciphertext letters.
- Letter frequencies are dispersed.
- Simple frequency analysis no longer works.

Key Space of the Vigenère Cipher

Suppose the key has length ℓ .

Each key position can be any of the 26 letters.

$$|\mathcal{K}| = 26^\ell$$

Key Length	Number of Keys
3	$26^3 = 17\,576$
5	$26^5 \approx 1.19 \times 10^7$
10	$26^{10} \approx 1.41 \times 10^{14}$

A longer key significantly increases the key space.

Why Was the Vigenère Cipher Eventually Broken?

Although frequency analysis is hidden,

the key repeats periodically.

Repeated key

$\underbrace{LEMON}_{\text{key}} \underbrace{LEMON}_{\text{repeats}} \underbrace{LEMON}_{\text{repeats}}$

creates repeated encryption patterns in long messages.

Once the key length is discovered,

- split the ciphertext into ℓ groups,
- each group becomes a Caesar cipher,
- apply frequency analysis to each group separately.

Kasiski and Friedman Attacks

Two classical cryptanalytic techniques estimate the key length.

Kasiski Examination

- Find repeated ciphertext fragments.
- Measure distances between repetitions.
- Compute the greatest common divisor (GCD) of these distances.
- The GCD often reveals the key length.

Friedman Test

- Measure the *Index of Coincidence (IC)* of the ciphertext.
- Estimate the key length statistically.

After estimating the key length, frequency analysis breaks each Caesar cipher independently.

One-Time Pad

Replace the repeating keyword with a truly random key.

- Message: $m \in \{0, 1\}^\ell$
- Key: $k \xleftarrow{\$} \{0, 1\}^\ell$, where $|k| = |m|$
- Encryption:

$$c = m \oplus k$$

- Decryption:

$$m = c \oplus k.$$

Advantages

- Perfect secrecy.
- Information-theoretically secure.

Problem

- Key must be random.
- Key length equals message length.
- Key can never be reused.

From Classical to Modern Cryptography

Classical Problem	Modern Solution
Small key space	Large computational keys
Frequency leakage	Computational security
Key reuse	Secure modes of operation
Long random keys	Pseudorandom generators
Key distribution	Public-key cryptography

Modern cryptography preserves the intuition of classical ciphers while providing mathematically provable security.

Block Cipher Based Encryption

A block cipher is a family of keyed permutations.

$$E_k : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

Examples:

- AES (128-bit block)
- DES (64-bit block)

To encrypt long messages, a block cipher is used with a **mode of operation**, such as

- **ECB** (Electronic Codebook) (Not recommended)
- **CBC** (Cipher Block Chaining)
- **CTR** (Counter Mode)
- **GCM** (Galois/Counter Mode) (Authenticated Encryption)

How Block Cipher Based Encryption Works

A block cipher encrypts one fixed-size block using a secret key.

$$\boxed{m_i} \xrightarrow{E_k} \boxed{c_i}$$

where

$$m_i, c_i \in \{0, 1\}^n.$$

For a longer message,

$$m = m_1 \| m_2 \| \dots \| m_t,$$

each block is encrypted using the same secret key together with a **mode of operation**.

$$m_1, m_2, \dots, m_t \xrightarrow{\text{AES} + \text{Mode}} c_1, c_2, \dots, c_t$$

Modes of Operation

A block cipher encrypts only one block. A mode of operation securely encrypts arbitrary-length messages.

Mode	Main Idea
ECB	Encrypt blocks independently
CBC	Chain neighboring blocks
CTR	Encrypt a counter to generate a keystream
GCM	CTR mode + authentication

- Modern systems primarily use CTR and GCM.
- ECB is generally insecure for encrypting multiple blocks.

We will study in details later.

OTP vs. Block Cipher Encryption

	One-Time Pad	Block Cipher
Security	Perfect	Computational
Key Length	$ k = m $	Fixed (128/192/256 bits)
Key Reuse	Never	Allowed
Practical?	No	Yes
Examples	OTP	AES, DES

Today, almost all practical symmetric encryption is based on block ciphers or stream ciphers.

How a Stream Cipher Works

A stream cipher encrypts one bit (or byte) at a time using a pseudorandom keystream.

$$k \xrightarrow{\text{Keystream Generator}} z_1, z_2, \dots, z_\ell$$

$$m_1, m_2, \dots, m_\ell$$

$$\oplus$$

$$z_1, z_2, \dots, z_\ell$$

$$\Downarrow$$

$$c_1, c_2, \dots, c_\ell$$

Encryption:

$$c_i = m_i \oplus z_i$$

Decryption:

$$m_i = c_i \oplus z_i$$

Examples: **ChaCha20**, **Salsa20**, **RC4 (deprecated)**

Generating the Keystream

A stream cipher expands a short secret key into a long pseudorandom keystream.

$$k \xrightarrow{\text{PRG}} z_1 z_2 \cdots z_\ell$$

where

- k is a short secret key.
- PRG is a **Pseudorandom Generator**.
- $z_1 z_2 \cdots z_\ell$ is the keystream.

The keystream should be computationally indistinguishable from a truly random bit string.

One-Time Pad vs. Stream Cipher

Property	One-Time Pad	Stream Cipher
Keystream	Truly random	Pseudorandom
Key Length	Same as message	Short secret key
Keystream Generation	Random key material	Generated by a PRG
Key Reuse	Never	New nonce/IV with same key
Security	Information-theoretic	Computational
Practicality	Impractical for large data	Highly practical

Both encrypt using

$$c = m \oplus z,$$

where z is the keystream. The difference lies in how the keystream is generated.

Establishing Short Key

Establishing a Shared Secret Key

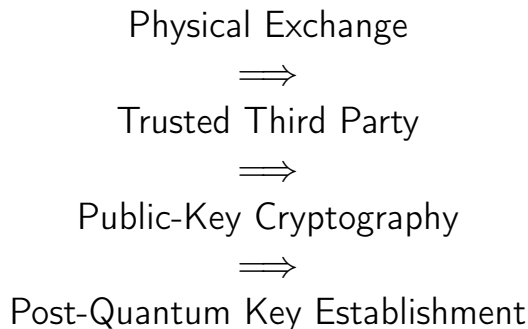
Encryption is easy once Alice and Bob share a secret key.

The real challenge is:

How can two parties establish a shared secret over a public network?

The answer has evolved significantly over time.

Evolution of Key Establishment



Each generation was developed to address new technological and computational challenges.

■ Physical Key Exchange

- Secret key delivered in person or through a trusted courier.

■ Trusted Third Party

- A trusted authority generates and distributes secret keys.

■ Limitation

- Difficult to scale for large communication networks.

Public-key cryptography enables two parties to establish a shared secret over an insecure channel.

Representative key establishment protocols:

- Diffie-Hellman Key Exchange (1976)
- Elliptic Curve Diffie-Hellman (ECDH)

Security relies on computational hardness assumptions, such as:

- Discrete Logarithm Problem
- Elliptic Curve Discrete Logarithm Problem

Large-scale quantum computers threaten many classical public-key systems.

Shor's algorithm efficiently solves:

- Integer Factorization
- Discrete Logarithm

Consequently,

RSA, Diffie-Hellman, ECDH

are no longer considered quantum-secure.

Post-Quantum Key Establishment

Modern key establishment is based on problems believed to resist quantum attacks.

Examples:

- Module-Lattice-Based Key Encapsulation Mechanisms (ML-KEM/Kyber)
- Code-Based Cryptography
- Isogeny-Based Cryptography (research)

Shared Secret

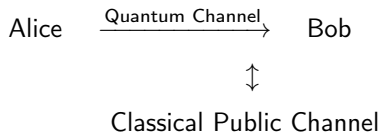


Symmetric Encryption (AES/ChaCha20)

This hybrid design is used in modern secure communication protocols.

Quantum Key Distribution (QKD)

Unlike classical key establishment, QKD derives its security from the laws of quantum mechanics.



- Uses quantum states (typically single photons) to establish a shared secret key.
- Any eavesdropping attempt introduces detectable disturbances.
- The generated key is subsequently used with symmetric encryption (e.g., AES or One-Time Pad).

QKD: Advantages and Limitations

Advantages

- Information-theoretic security.
- Detects eavesdropping.
- Secure against quantum computers.

Limitations

- Requires specialized quantum hardware.
- Limited communication distance and key generation rate.
- Does not encrypt data; it only establishes secret keys.

QKD is a key establishment protocol, not an encryption scheme.

Achieving Integrity

Confidentiality hides the message.

Integrity ensures that the message has **not been modified**.

Examples of integrity violations:

- Changing the amount in a bank transaction.
- Modifying a software update.
- Altering a medical record.

How Do We Achieve Integrity?

A cryptographic checksum is attached to every message.

$$m \xrightarrow{\text{Integrity Algorithm}} t$$

Sender transmits

$$(m, t)$$

Receiver recomputes the tag.

$$t \stackrel{?}{=} t'$$

If they match, the message is accepted; otherwise, it is rejected.

Techniques for Achieving Integrity

Technique	Purpose
Hash Function	Detect accidental modifications
Message Authentication Code (MAC)	Detect malicious modifications
Digital Signature	Integrity + authentication + non-repudiation

These techniques will be studied in later lectures.

Achieving Authenticity

Authentication answers the question:

Who created this message?

Authentication assures that the sender is genuine.

Examples:

- Online banking
- Software updates
- Secure email
- Digital certificates

How Do We Achieve Authentication?

Authentication requires cryptographic evidence that only the legitimate sender can generate.

$$m \xrightarrow{\text{Authentication Algorithm}} a$$

Sender transmits

$$(m, a)$$

Receiver verifies

$$\text{Verify}(m, a)$$

Accepted only if the proof is valid.

Techniques for Achieving Authentication

Technique	Used In
Message Authentication Code (MAC)	Shared-secret systems
Digital Signature	Public-key systems
Digital Certificates	Identity verification

A MAC provides authentication only between parties sharing the same secret key, whereas digital signatures provide public verifiability.

Integrity vs Authentication

Integrity
Authentication

Has the message been modified?
Who generated the message?

Hash Function



MAC



Digital Signature

These primitives together provide secure and trustworthy communication.

What Comes Next?

We have learned *why* encryption is needed and *how* secret keys can be established.

Next, we study the mathematical foundations of modern cryptography:

- 1 Pseudorandom Generators (PRGs)
- 2 Pseudorandom Functions (PRFs)
- 3 Pseudorandom Permutations (PRPs)
- 4 Block Ciphers and Stream Ciphers
- 5 Secure Modes of Operation
- 6 Formal Security Definitions

One-Way Functions

$\implies$ PRGs

$\implies$ PRFs / PRPs

$\implies$ Encryption

$\implies$ Authenticated Encryption

$\implies$ Public-Key Cryptography

$\implies$ Post-Quantum Cryptography

From computational hardness to secure communication.

Textbook:

- 1 Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. 4th Edition. Chapman & Hall/CRC Press, 2019. ISBN: 9780367148591.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram

`laltu.sardar@iisertvm.ac.in`, `laltu.sardar.crypto@gmail.com`

Course webpage: https://laltu-sardar.github.io/courses/corgos_2026.html.